

WORLDLINE GLOBAL

Round 2 - SDET / Test Automation Lead

MASTER INTERVIEW PREPARATION PLAN

1.5-hour technical round | Coding | Debugging | API | Concurrency | Test Design | Automation | Engineering

Primary objective	Demonstrate strong individual technical execution, structured problem solving, defensive coding, test design, debugging and SDET engineering judgement.
Likely style	Hands-on and diagnostic: code review, bug finding, coding, test-case design and deep follow-up questions.
Golden rule	Do not give restricted answers. Explain your reasoning, identify edge cases, ask useful clarifying questions and verify assumptions.

How to use this PDF: First study the roadmap and question bank. Then we will work question-by-question: you answer first, I challenge, correct and give you an interview-ready version.

1. What the feedback tells us

Feedback signal	What it likely means
Defensive behaviour / restricted response	They evaluate communication and reasoning under pressure, not only factual knowledge.
API testing + mocking	Expect practical unit/integration testing, external dependency failure and Mockito/WireMock scenarios.
Third-party unavailable	Expect timeout, 5xx, retry, fallback, circuit-breaker and mocking.
Debugging / exception handling	Likely buggy-code or code-review exercise.
NullPointerException	They expect proactive defensive coding rather than try/catch-only answers.
Concurrency / race condition	Expect shared-state coding and test design.
Floating-point precision	Expect boundary/data-type reasoning; BigDecimal is relevant.
Allure / Extent	Know implementation and CI artifact flow, not just tool names.
Spring Boot	Know controller/service/repository and backend test strategy.
PUT / DELETE / idempotency	Know HTTP semantics and how to design idempotency tests.
Lack of curiosity	Ask clarifying questions, explore edge cases and explain thinking.
Selected candidate: 4/5 bugs	Prepare a timed debugging exercise with multiple hidden defects.
Selected candidate: synchronized	Prepare race-condition code and synchronized vs AtomicInteger.
Selected candidate: 9 test cases	Practise balanced positive/negative/boundary test design.

Bottom line: this is likely an **engineering + SDET problem-solving round**, not a pure Selenium interview.

2. Priority map

Priority	Area	Target
MASTER	Java coding + debugging	Find defects, write clean code, explain trade-offs
MASTER	Test-case design	Positive, negative, boundary, concurrency, precision
MASTER	API + mocking	Rest Assured, Mockito, third-party failures
MASTER	Concurrency	Race conditions, synchronized, AtomicInteger
MASTER	Exceptions	NPE, Error vs Exception, propagation, defensive coding
MASTER	HTTP semantics	PUT/DELETE idempotency, POST, status codes
PRACTISE	Allure / Extent	Implementation + CI artifact/report flow
PRACTISE	Spring Boot	Controller/service/repository + testing
PRACTISE	Selenium / Playwright	Framework + migration story
PRACTISE	JUnit / Mockito	Unit testing and verification
MASTER	Communication	Curiosity, ownership, reasoning under pressure

3. Java coding, debugging & core language - MASTER

Practise coding while talking. When shown code, inspect nulls, zero/negative values, boundaries, precision, shared state, exceptions and resource lifecycle.

#	Question / Topic	Priority
1	Write second-highest distinct number using Streams.	MASTER
2	Find duplicates in an array using Streams.	MASTER
3	First non-repeated character.	MASTER
4	Word/character frequency using Map.	MASTER
5	Remove duplicates while preserving insertion order.	MASTER
6	List vs Set vs Map with automation examples.	MASTER
7	HashMap internals and collisions.	PRACTISE
8	equals() vs hashCode().	MASTER

#	Question / Topic	Priority
9	Comparable vs Comparator.	MASTER
10	map vs flatMap vs filter vs reduce.	MASTER
11	Optional: findFirst, orElse, orElseGet, orElseThrow.	MASTER
12	Checked vs unchecked exceptions.	MASTER
13	Exception vs Error.	MASTER
14	How to prevent NullPointerException?	MASTER
15	When catch vs propagate an exception?	MASTER
16	Custom exception design.	PRACTISE
17	Immutable class design.	PRACTISE
18	final vs finally vs finalize.	PRACTISE
19	synchronized keyword and critical section.	MASTER
20	Create and fix a race condition.	MASTER
21	synchronized vs AtomicInteger.	MASTER
22	ThreadLocal and automation.	MASTER

4. Test-case design - MASTER

Target: produce a balanced set, not a random list. The selected candidate's feedback strongly suggests this is scored.

#	Question / Topic	Priority
1	How do you design tests from a short requirement?	MASTER
2	Positive vs negative vs boundary scenarios.	MASTER
3	Design cases for min/max constrained field.	MASTER
4	Amount: null, 0, negative, min, max, max+1, precision.	MASTER
5	Concurrency: two users buy the last item.	MASTER
6	Race condition: two updates to same balance.	MASTER
7	Floating-point / monetary calculation tests.	MASTER
8	Duplicate API request tests.	MASTER
9	Third-party unavailable tests.	MASTER
10	Timeout / slow response tests.	MASTER
11	400/401/403/404/409/429/500/502/503/504.	MASTER
12	Authentication and authorisation tests.	MASTER
13	PUT idempotency tests.	MASTER
14	DELETE idempotency tests.	MASTER
15	Pagination tests.	PRACTISE
16	Search/filter/sort tests.	PRACTISE
17	File upload/download tests.	PRACTISE
18	How to prioritise under time pressure?	MASTER
19	How to decide what not to automate?	PRACTISE

5. API testing + mocking - MASTER

Think: mock/stub, timeout, 5xx, malformed response, retry, duplicate side effects, logging and correct business response.

#	Question / Topic	Priority
1	REST API end-to-end testing approach.	MASTER
2	Status code, headers and response validation.	MASTER
3	Path vs query vs body.	MASTER
4	POST vs PUT vs PATCH.	MASTER
5	PUT idempotency.	MASTER
6	DELETE idempotency.	MASTER
7	Is POST idempotent?	MASTER
8	Does idempotent mean same response?	MASTER
9	Third-party server down: how do you test?	MASTER
10	How do you mock a third-party API?	MASTER
11	Mockito mock vs spy.	MASTER
12	thenReturn vs thenThrow.	MASTER
13	verify() and verification.	MASTER
14	ArgumentCaptor.	PRACTISE
15	Timeout from external dependency.	MASTER
16	HTTP 503 dependency failure.	MASTER
17	Retry and retry exhaustion.	PRACTISE
18	Circuit breaker and how to test it.	PRACTISE
19	WireMock vs Mockito.	MASTER
20	Unit vs integration vs contract vs E2E.	MASTER
21	JSON Schema validation.	MASTER
22	POJO serialization/deserialization.	MASTER
23	Basic/Bearer/OAuth2/JWT.	MASTER
24	API chaining.	PRACTISE
25	Negative API testing.	MASTER
26	Dynamic response values.	PRACTISE

6. JUnit + Mockito - MASTER

#	Question / Topic	Priority
1	JUnit 5 lifecycle annotations.	MASTER
2	JUnit assertions.	MASTER
3	Parameterized tests.	PRACTISE
4	@Mock, @InjectMocks, @Spy, @Captor.	MASTER
5	Mock successful external response.	MASTER
6	Mock dependency exception.	MASTER
7	verify once / never.	MASTER
8	Argument matchers.	MASTER
9	When should you avoid mocking?	MASTER
10	How to avoid over-mocking?	PRACTISE
11	Unit test service + repository + external client.	MASTER
12	Test exception mapping/propagation.	MASTER

#	Question / Topic	Priority
13	Test null and invalid inputs.	MASTER
14	What makes a good unit test?	MASTER

7. Concurrency, race conditions & thread safety - MASTER

#	Question / Topic	Priority
1	What is a race condition?	MASTER
2	What is thread safety?	MASTER
3	Why is count++ not atomic?	MASTER
4	Write a counter with a race condition.	MASTER
5	Fix with synchronized.	MASTER
6	Fix with AtomicInteger.	MASTER
7	synchronized method vs block.	MASTER
8	Monitor/lock.	PRACTISE
9	Deadlock and prevention.	MASTER
10	volatile vs synchronized.	MASTER
11	ThreadLocal in automation.	MASTER
12	Why static WebDriver is dangerous in parallel?	MASTER
13	Playwright BrowserContext isolation.	MASTER
14	Concurrency test for inventory.	MASTER
15	Concurrent duplicate payment test.	MASTER
16	Concurrent account-balance test.	MASTER
17	HashMap concurrency.	PRACTISE
18	ConcurrentHashMap vs HashMap.	PRACTISE
19	ExecutorService basics.	PRACTISE

8. Exception handling + defensive coding - MASTER

#	Question / Topic	Priority
1	Exception vs Error.	MASTER
2	Checked vs unchecked.	MASTER
3	NPE causes.	MASTER
4	Prevent NPE instead of catching it.	MASTER
5	Should you catch NPE?	MASTER
6	throw vs throws.	MASTER
7	Catch vs propagate.	MASTER
8	Preserve root cause.	MASTER
9	Why catch(Exception) is risky.	MASTER
10	Why swallowing exceptions is dangerous.	MASTER
11	finally and try-with-resources.	PRACTISE
12	Custom exception hierarchy.	PRACTISE
13	Defensive validation: null/zero/negative/max.	MASTER
14	Malformed external response.	MASTER

#	Question / Topic	Priority
15	Timeout vs business failure.	MASTER

9. Floating-point precision & numeric correctness - MASTER

#	Question / Topic	Priority
1	Why can $0.1 + 0.2$ be imprecise?	MASTER
2	Why is double risky for money?	MASTER
3	When use BigDecimal?	MASTER
4	BigDecimal(String) vs BigDecimal(double).	MASTER
5	Scale and rounding mode.	PRACTISE
6	Tax/price calculation tests.	MASTER
7	Decimal boundaries.	MASTER
8	Safe floating-point comparison.	MASTER
9	Overflow/underflow.	PRACTISE

10. REST semantics + Spring Boot backend - MASTER/PRACTISE

#	Question / Topic	Priority
1	Controller → Service → Repository architecture.	MASTER
2	@RestController / @Service / @Repository.	MASTER
3	Spring dependency injection.	MASTER
4	Constructor vs field injection.	PRACTISE
5	Unit test a Spring service.	MASTER
6	Mock repository.	MASTER
7	Mock external REST client.	MASTER
8	@ControllerAdvice/global exception handling.	PRACTISE
9	Validation errors.	MASTER
10	400 vs 401 vs 403.	MASTER
11	404 vs 409.	MASTER
12	429.	MASTER
13	500 vs 502 vs 503 vs 504.	MASTER
14	PUT/DELETE idempotency.	MASTER
15	POST vs PUT vs PATCH.	MASTER
16	Optimistic locking/version conflicts.	PRACTISE

11. Reporting - Allure / Extent - PRACTISE TO MASTER

#	Question / Topic	Priority
1	How did you implement Allure?	MASTER
2	Where are Allure results generated?	MASTER
3	How do you attach screenshots?	MASTER
4	Attach API request/response.	MASTER
5	Attach logs.	PRACTISE
6	Publish Allure from Jenkins.	MASTER

#	Question / Topic	Priority
7	What artifacts do you retain after failure?	MASTER
8	How do you prevent report bloat?	PRACTISE
9	Allure vs Extent.	PRACTISE
10	How would you abstract reporting from the framework?	PRACTISE

12. Selenium + Playwright framework - PRACTISE/MASTER

#	Question / Topic	Priority
1	Why Selenium → Playwright?	MASTER
2	Migration strategy.	MASTER
3	What did you redesign instead of one-to-one conversion?	MASTER
4	Selenium waits vs Playwright auto-waiting.	MASTER
5	Did you remove all explicit waits?	MASTER
6	Locator strategy and data-testid.	MASTER
7	POM in Playwright.	MASTER
8	Browser vs BrowserContext vs Page.	MASTER
9	Parallel execution.	MASTER
10	Fixtures/lifecycle.	MASTER
11	Authentication state.	PRACTISE
12	Network interception/mocking.	MASTER
13	Tracing/screenshots/video.	PRACTISE
14	Debugging flaky tests.	MASTER
15	Jenkins integration.	MASTER
16	How measured success?	MASTER
17	Biggest challenges.	MASTER
18	What did YOU own vs team?	MASTER
19	Why not continue Selenium?	MASTER

13. AI-assisted migration - MASTER

#	Question / Topic	Priority
1	How did you use Cursor?	MASTER
2	How did you use Claude?	MASTER
3	What did AI accelerate?	MASTER
4	What did AI not decide?	MASTER
5	Repository rules/skills.	MASTER
6	Concrete AI-assisted migration example.	MASTER
7	How did you validate AI-generated code?	MASTER
8	Prevent hallucinated APIs.	MASTER
9	Prevent hard waits.	MASTER
10	Locator quality with AI.	MASTER
11	Did AI write tests end-to-end?	MASTER
12	How measure productivity gain?	PRACTISE

#	Question / Topic	Priority
13	Risks of AI-generated automation code.	MASTER
14	Where should engineers not rely on AI?	MASTER

Key line: “AI accelerated implementation; it did not replace engineering decisions or validation.”

14. Automation design patterns - PRACTISE

#	Question / Topic	Priority
1	Why POM?	MASTER
2	Is POM a design pattern?	PRACTISE
3	Factory for browser creation.	MASTER
4	Builder for complex test data.	MASTER
5	Strategy in automation.	MASTER
6	Singleton: when appropriate?	MASTER
7	Why not Singleton WebDriver in parallel?	MASTER
8	Dependency injection / fixtures.	MASTER
9	Factory vs Strategy.	MASTER
10	Composition vs inheritance in Page Objects.	PRACTISE
11	Patterns used during Selenium → Playwright migration.	MASTER

15. Framework + CI/CD + DevOps - PRACTISE

#	Question / Topic	Priority
1	Explain framework architecture.	MASTER
2	Environment configuration.	MASTER
3	Secrets management.	MASTER
4	Smoke vs regression.	MASTER
5	CI test selection.	PRACTISE
6	Parallel execution.	MASTER
7	Test data management.	MASTER
8	Shared state prevention.	MASTER
9	Flaky tests in CI.	MASTER
10	Retries without hiding defects.	MASTER
11	Jenkins pipeline.	MASTER
12	Reports/artifacts.	MASTER
13	Git/Bitbucket PR validation.	PRACTISE
14	Docker for automation.	PRACTISE
15	Scaling execution.	PRACTISE

16. Leadership, ownership & curiosity - MASTER

#	Question / Topic	Priority
1	What exactly did you own in migration?	MASTER
2	How divided work?	PRACTISE
3	How reviewed automation code?	MASTER

#	Question / Topic	Priority
4	How established coding standards?	MASTER
5	How handled design disagreement?	PRACTISE
6	Migration priorities.	MASTER
7	Communicating risk.	PRACTISE
8	Measuring automation effectiveness.	MASTER
9	Difficult release defect.	PRACTISE
10	Flaky test you fixed.	MASTER
11	Framework failure and improvement.	MASTER
12	What would you improve?	MASTER
13	What would you do differently?	MASTER
14	What are you learning now and why?	MASTER
15	What if you don't know an answer?	MASTER

17. Likely hands-on exercises

Exercise A - Buggy Java method — Find 5-8 issues: null, zero, negative, precision, exception, shared state.

Exercise B - Race-condition counter — Reproduce lost updates; fix with synchronized and AtomicInteger; explain trade-offs.

Exercise C - Payment service — Mock success, timeout, 503 and malformed third-party response.

Exercise D - API test design — Produce 9 balanced positive/negative/boundary/concurrency cases.

Exercise E - Idempotency — Repeat PUT/DELETE and verify final state and unintended side effects.

Exercise F - Debugging — Classify root cause as product, test, environment or dependency.

Exercise G - Reporting — Explain screenshot/log/request-response attachment and Jenkins publication.

Exercise H - Spring unit test — Mock repository + external client; test success and failure paths.

Exercise I - Precision — Replace unsafe monetary double calculation with BigDecimal and tests.

18. Seven-day focused plan

Day	Focus	Output
Day 1	Java + exceptions + NPE + defensive coding	10 coding/debugging problems aloud
Day 2	API + Mockito + JUnit + dependency failures	5 unit tests with mocks
Day 3	Test design + boundaries + precision	9 cases for 5 requirements
Day 4	Concurrency + race conditions + idempotency	synchronized/AtomicInteger + concurrency tests
Day 5	Playwright migration + framework + reporting	90-second migration story + architecture
Day 6	Spring Boot + CI/CD + AI-assisted engineering	Backend/AI questions without notes
Day 7	Full mock round	90-minute simulation + fix weak areas

If fewer days are available, keep Days 1-4 non-negotiable and compress Days 5-7.

19. Interview-day checklist

- Talk through your reasoning while solving; do not go silent.
- Ask useful clarifying questions before coding.
- Check null, zero, negative, empty, min, max and duplicate cases.
- For concurrency, explicitly identify shared mutable state.
- For money, think BigDecimal and rounding.
- For external APIs, think mock/stub, timeout, 5xx, retry and duplicate side effects.
- For PUT/DELETE, explain idempotency in terms of final resource state.
- For exceptions, prefer prevention/validation over catch-all handling.
- For reporting, explain implementation and CI artifacts, not only tool names.
- For Playwright, be precise about Browser, BrowserContext, Page, fixtures and auto-waiting.
- For AI, explain rules, validation, review and where human judgement remains.
- If you do not know, reason openly instead of becoming defensive.
- Show curiosity by asking what the expected business behaviour is and challenging your own solution.

20. How we will prepare question-by-question

Method: I will act as the interviewer. You answer first. Then I will challenge your answer with follow-ups, correct gaps, show the clean implementation, give an interview-ready response, and move to the next question. We will deliberately practise the pressure points identified in the feedback.

Recommended starting point: Java debugging + defensive coding, followed by API mocking, test-case design, concurrency and idempotency.